

지방기능경기대회 채점기준

1. 채점 시 유의사항

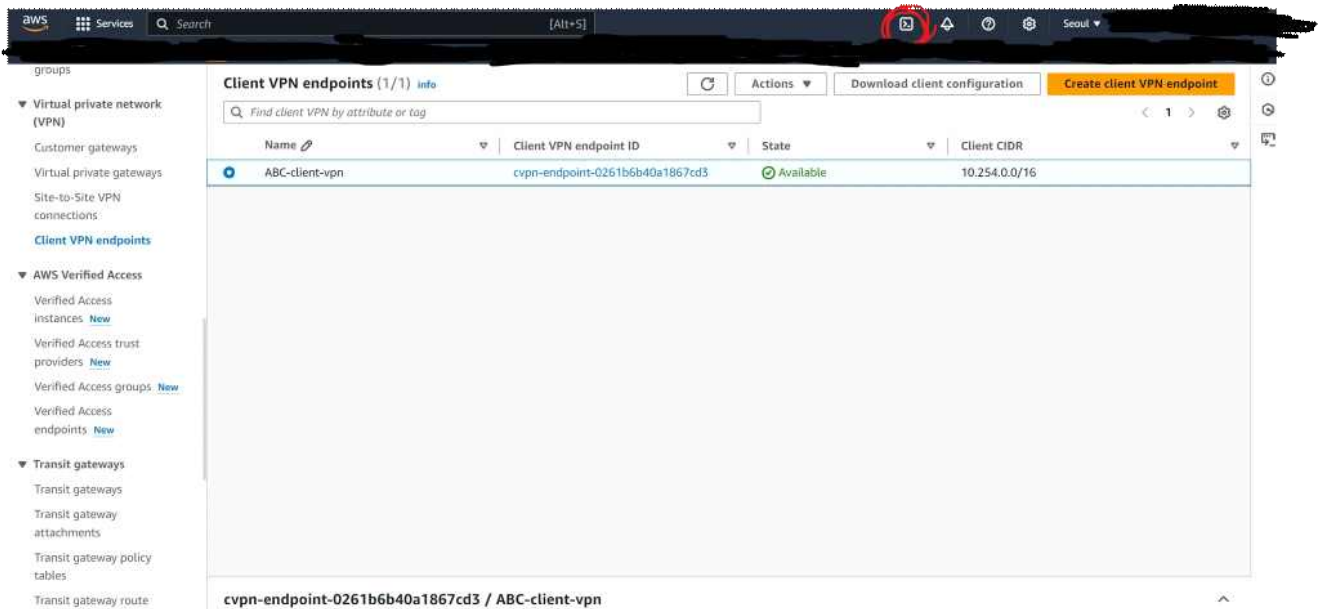
직 종 명

클라우드컴퓨팅

※ 다음 사항을 유의하여 채점하시오.

채점하는 서버가 선수의 서버가 맞는지 확인합니다.

채점시 명령어 입력은 아래 사진과 같이 CloudShell을 이용할 수 있습니다.



※ 채점기준 양식은 반드시, 양식에 맞추어 작성해야 합니다.(채점사이트 입력에 필요)

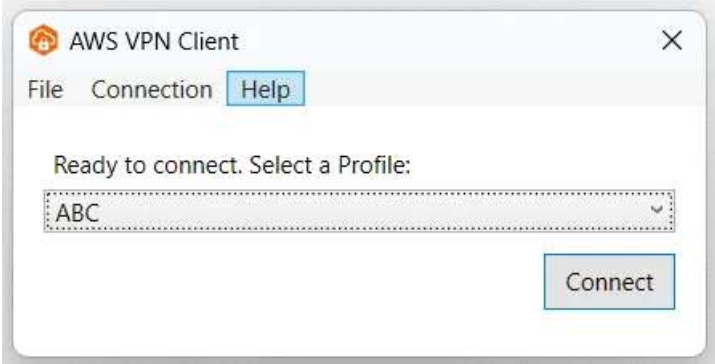
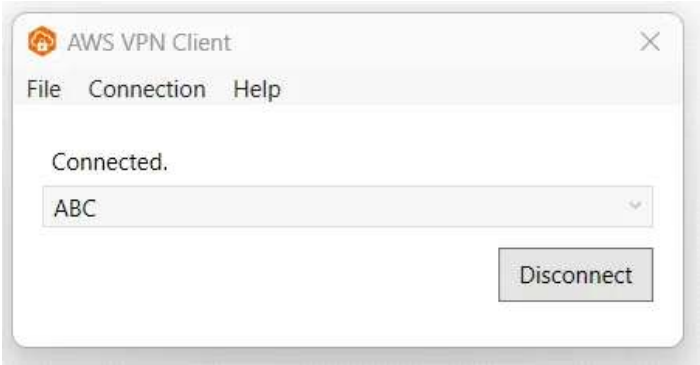
2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제2과제	1	VPN	12.5	○			○	
	2	Keda	12.5	○			○	
	3	Terraform	12.5	○			○	
	4	EC2	12.5	○			○	
합 계			50					

2) 채점방법 및 기준

과제 번호	일 련 번 호	주요항목	일련 번호	세부항목(채점방법)	배점
제2과제	1	VPN	1	VPC 생성	1.5
			2	Public Subnet CIDR	1
			3	Private Subnet CIDR	1
			4	NAT GW	1.5
			5	Client VPN CIDR	1.5
			6	Client VPN HA	1.5
			7	Client VPN ACL	1.5
			8	Client VPN 연결 확인	1.5
			9	Client VPN 서버 접속	1.5
	2	Evnet-driven Autoscaling	1	EKS cluster 생성	1
			2	Deployment 생성	1.5
			3	Deployment 최소 갯수	1.5
			4	Deployment 환경 변수	1.5
			5	SQS queue 생성	1
			6	Keda 생성	1.5
			7	Scaled Object	1.5
			8	컨테이너 확장	1.5
			9	컨테이너 축소	1.5
	3	Infrastructure as Code	0	사전작업	-
			1	EC2 생성	1
			2	VPC 생성	1
			3	Public subnet 생성	1.5
			4	IGW 생성	1.5
			5	EC2 타입	1.5
			6	EC2 서브넷	1.5
			7	EBS 암호화	1.5
			8	IAM role	1.5
			9	SSM 연결	1.5
	4	EC2 Automation	1	테스트 인스턴스 생성	1
			2	old 스크립트 확인 및 실행	1.5
			3	인스턴스1 삭제	1.5
			4	인스턴스2 삭제	1.5
			5	인스턴스3,4 미삭제	1.5
			6	all 스크립트 확인 및 실행	1
			7	인스턴스3 삭제	1.5
			8	인스턴스4 삭제	1.5
			9	Bastion 구동 확인	1.5
합계					50

순번	채점 항목
1-1	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ export VPC_ID=\$(aws ec2 describe-vpcs --filters W Name=tag:Name,Values=[ws-vpn-vpc] --query Vpcs[].VpcId --output text) \$ aws ec2 describe-vpcs --filters Name=tag:Name,Values=[ws-vpn-vpc] W --query Vpcs[].CidrBlock</pre> 3) 출력되는 결과가 "10.99.0.0/16" 인지 확인 합니다.
1-2	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ aws ec2 describe-subnets --filters W Name=tag:Name,Values=[ws-public-subnet-a] --query Subnets[].CidrBlock</pre> 3) 출력되는 결과가 "10.99.0.0/24" 인지 확인 합니다.
1-3	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ aws ec2 describe-subnets --filters W Name=tag:Name,Values=[ws-private-subnet-b] --query Subnets[].CidrBlock</pre> 3) 출력되는 결과가 "10.99.11.0/24" 인지 확인 합니다.
1-4	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ export PUBLIC_SUBNET_IDS=\$(aws ec2 describe-nat-gateways --filter W Name=state,Values=[available] --query NatGateways[].SubnetId --output text) \$ aws ec2 describe-subnets --subnet-ids \$PUBLIC_SUBNET_IDS --query W Subnets[].AvailabilityZone</pre> 3) 출력되는 결과에 "ap-northeast-2a", "ap-northeast-2b" 모두 포함 되어 있는지 확인합니다.
1-5	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ export CVPN_ID=\$(aws ec2 describe-client-vpn-endpoints --filters W Name=tag:Name,Values=[ws-client-vpn] --query W ClientVpnEndpoints[].ClientVpnEndpointId --output text) \$ aws ec2 describe-client-vpn-endpoints --client-vpn-endpoint-id \$CVPN_ID W --query ClientVpnEndpoints[].ClientCidrBlock</pre> 3) 출력되는 결과가 "10.254.0.0/16" 인지 확인 합니다.

순번	채점 항목
1-6	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ aws ec2 describe-client-vpn-target-networks --client-vpn-endpoint-id W \$CVPN_ID --query ClientVpnTargetNetworks[].TargetNetworkId --output text</pre> <pre>\$ aws ec2 describe-instances --filters W Name=tag:Name,Values=[ws-web-a,ws-web-b] W --query Reservations[].Instances[].SubnetId --output text</pre> 3) 위에서 입력한 두개의 명령어의 출력 결과가 동일한지 확인합니다. * 결과값이 동일하고 순서만 다른 경우에도 정답으로 인정합니다.
1-7	1) 웹브라우저로 선수의 AWS 계정에 로그인 후 CloudShell을 실행합니다. 2) 아래 명령어를 입력 합니다. <pre>\$ aws ec2 describe-client-vpn-authorization-rules --client-vpn-endpoint-id W \$CVPN_ID --query AuthorizationRules[].DestinationCidr</pre> 3) 출력되는 결과가 "10.99.0.0/16" 인지 확인 합니다.
1-8	1) 선수의 윈도우 PC에서 AWS VPN Client를 실행하고 선수가 미리 구성해둔 프로필을 선택해 연결(Connect)를 시도 합니다.  2) 아래 그림 처럼 연결에 성공해 상태가 "Conntected."로 변경 되는지 확인합니다. 
1-9	1) 1-8 항목에서 VPN 연결을 성공해 VPC와 연결된 상태에서 진행합니다 2) VPC에 생성한 웹서버의 Private IP 주소를 기록합니다. 3) 브라우저에서 http://<EC2 웹서버 주소> 를 입력합니다. 4) "Welcome to the ws portal" 이라고 보이는지 확인합니다.

순번	채점 항목
2-1	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ aws eks describe-cluster --name order-cluster --query cluster.version 3) 1.31이 출력 되는지 확인합니다.
2-2	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ aws eks update-kubeconfig --name order-cluster \$ kubectl get deployment --namespace order grep order-processor 3) order-processor가 출력 되는지 확인합니다.
2-3	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ kubectl get po -n order grep order-processor 3) 하나의 order-processor Pod만 출력 되는지 확인합니다.
2-4	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ kubectl describe deployment -n order grep -E "QUEUE_URL REGION_NAME" 3) QUEUE_URL과 REGION_NAME 환경 변수가 출력되는지 확인합니다.
2-5	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ aws sqs get-queue-url --queue-name order-queue --query QueueUrl W --output text 3) SQS의 URL이 출력 되는지 확인합니다.
2-6	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ kubectl get po -A grep keda-operator 3) keda-operator가 포함된 결과가 출력 되는지 확인합니다.
2-7	1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력합니다. \$ kubectl get scaledobjects.keda.sh -A 3) 1개 이상의 결과물(ScaledObjects)가 출력 되는지 확인합니다.
2-8	한대 Pod만 있어야 가능 함으로 2-3을 틀렸다면 2-8도 오답으로 간주합니다. 1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력하여 부하를 만듭니다. \$ for i {1..20}; do aws sqs send-message --queue-url <<선수 Q>> W --message-body \$i; done; 3) 아래 명령어를 약 5초 단위로 계속 입력해 Pod가 증설 되는지 확인합니다. \$ kubectl get po -n order grep order-processor (1대 이상 증설까지 최대 5분 대기 가능)

순번	채점 항목
2-9	Pod 감소 테스트 임으로 2-8을 틀렸다면 2-9도 오답으로 간주합니다. 1) Bastion의 shell에 접근합니다. 2) 아래 명령어를 약 5초 단위로 계속 입력해 Pod가 감소 되는지 확인합니다. <pre>\$ kubectl get po -n order grep order-processor</pre> (Pod 감소 까지 최대 5분 대기 가능)
3-0	사전작업 단계이며 해당 명령어를 실행하여 리소스를 삭제 후 재생성 합니다. 1) Default VPC에 배포된 Bastion의 /home/ec2-user/korea에 CLI로 접근합니다. 2) 아래 명령어를 입력하여 기존에 생성된 리소스를 삭제하고, 테라폼 코드 기반으로 인프라를 재생성합니다. <pre>\$ cd /home/ec2-user/korea \$ terraform init && terraform destroy -auto-approve \$ terraform apply -auto-approve</pre>
3-1	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 i- 로 시작하는 인스턴스 ID가 출력되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[korea-instance] W --query Reservations[].Instances[].InstanceId --output text</pre>
3-2	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 10.0.0.0/16이 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-vpcs --filters Name=tag:Name,Values=[korea-vpc] W --query Vpcs[].CidrBlock</pre>
3-3	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 ap-northeast-2a가 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-subnets --filters Name=tag:Name,Values=[korea-public-subnet-a] W --query Subnets[].AvailabilityZone</pre>
3-4	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 igw- 로 시작하는 IGW ID가 출력되는지 확인합니다. <pre>\$ export VPC_ID=\$(aws ec2 describe-vpcs --filters W Name=tag:Name,Values=[korea-vpc] --query Vpcs[].VpcId --output text) \$ aws ec2 describe-internet-gateways --filters W Name=attachment.vpc-id,Values=[\$VPC_ID] --query W InternetGateways[].InternetGatewayId</pre>
3-5	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 t3.micro가 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[korea-instance] W --query Reservations[].Instances[].InstanceType</pre>

순번	채점 항목
3-6	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 korea-public-subnet-a가 출력 되는지 확인합니다. <pre>\$ export SUBNET_ID=\$(aws ec2 describe-instances --filters Name=tag:Name,Values=[korea-instance] W --query Reservations[].Instances[].SubnetId) \$ aws ec2 describe-subnets --subnet-ids \$SUBNET_ID --query Subnets[].Tags[?Key==W`NameW`].Value</pre>
3-7	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 true 가 출력 되는지 확인합니다. <pre>\$ export VOLUME_ID=\$(aws ec2 describe-instances --filters W Name=tag:Name,Values=[korea-instance] --query W Reservations[].Instances[].BlockDeviceMappings[].Ebs.VolumId --output text) \$ aws ec2 describe-volumes --volume-ids \$VOLUME_ID --query Volumes[].Encrypted</pre>
3-8	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 IAM role의 ARN이 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[korea-instance] W --query Reservations[].Instances[].IamInstanceProfile grep -i Arn</pre>
3-9	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 Online 이 출력 되는지 확인합니다. <pre>\$ export INSTANCE_ID=\$(aws ec2 describe-instances --filters W Name=tag:Name,Values=[korea-instance] --query W Reservations[].Instances[].InstanceId --output text) \$ aws ssm describe-instance-information --filters W Key=InstanceIds,Values=[\$INSTANCE_ID] --query InstanceInformationList[].PingStatus</pre>

순번	채점 항목
4-1	1) Tokyo에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 에러없이 실행 되는지 확인합니다. <pre>\$ aws ec2 run-instances --image-id "ami-03f584e50b2d32776" --instance-type "t3.micro" W --tag-specifications W '{"ResourceType":"instance","Tags":[{"Key":"Name","Value":"instance-1-2025x2gj"}], W {"Key":"Project","Value":"skills2022"}]}' --count "1" --no-cli-pager > /dev/null</pre> <pre>\$ aws ec2 run-instances --image-id "ami-03f584e50b2d32776" --instance-type "t3.micro" W --tag-specifications W '{"ResourceType":"instance","Tags":[{"Key":"Name","Value":"instance-2-2025x2gj"}], W {"Key":"Project","Value":"skills2022"}]}' --count "1" --no-cli-pager > /dev/null</pre> <pre>\$ aws ec2 run-instances --image-id "ami-03f584e50b2d32776" --instance-type "t3.micro" W --tag-specifications W '{"ResourceType":"instance","Tags":[{"Key":"Name","Value":"instance-3-2025x2gj"}], W {"Key":"Project","Value":"skills2023"}]}' --count "1" --no-cli-pager > /dev/null</pre> <pre>\$ aws ec2 run-instances --image-id "ami-03f584e50b2d32776" --instance-type "t3.micro" W --tag-specifications W '{"ResourceType":"instance","Tags":[{"Key":"Name","Value":"instance-4-2025x2gj"}], W {"Key":"Project","Value":"skills2024"}]}' --count "1" --no-cli-pager > /dev/null</pre>
4-2	1) Tokyo에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어로 delete_old_instance.sh가 있는지 확인합니다. (존재 시 득점) <pre>\$ cd /home/ec2-user/ec2-automation/ \$ ls -l grep delete_old_instance</pre> 3) 존재 한다면 아래 명령어로 delete_old_instance를 실행합니다. <pre>\$ chmod +x delete_old_instance.sh \$./delete_old_instance.sh</pre> (약 30초 대기 후 4-3 채점을 시작 합니다.)
4-3	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 instance-1의 상태가 "Shutting-down" 이나 "Terminated"로 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[instance-1-2025x2gj] W --query Reservations[].Instances[].State</pre>
4-4	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 instance-2의 상태가 "Shutting-down" 이나 "Terminated"로 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[instance-2-2025x2gj] W --query Reservations[].Instances[].State</pre>

순번	채점 항목
4-5	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 instance-3과 4의 상태가 "Running" 인지 확인합니다. <pre>\$ aws ec2 describe-instances --filters W Name=tag:Name,Values=[instance-3-2025x2gj] --query Reservations[].Instances[].State</pre> <pre>\$ aws ec2 describe-instances --filters W Name=tag:Name,Values=[instance-4-2025x2gj] --query Reservations[].Instances[].State</pre>
4-6	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어로 delete_all_instance.sh가 있는지 확인합니다. (존재 시 득점) <pre>\$ cd /home/ec2-user/ec2-automation/ \$ ls -l grep delete_all_instance</pre> 3) 존재 한다면 아래 명령어로 delete_all_instance를 실행합니다. <pre>\$ chmod +x delete_all_instance.sh \$./delete_all_instance.sh</pre> (약 30초 대기 후 4-7 채점을 시작 합니다.)
4-7	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 instance-3의 상태가 "Shutting-down" 이나 "Terminated"로 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[instance-3-2025x2gj] W --query Reservations[].Instances[].State</pre>
4-8	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 instance-4의 상태가 "Shutting-down" 이나 "Terminated"로 출력 되는지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[instance-4-2025x2gj] W --query Reservations[].Instances[].State</pre>
4-9	1) Default VPC에 배포된 Bastion의 shell에 접근합니다. 2) 아래 명령어를 입력해 bastion 서버의 상태가 "Running" 인지 확인합니다. <pre>\$ aws ec2 describe-instances --filters Name=tag:Name,Values=[automation-bastion] W --query Reservations[].Instances[].State</pre>